

基于强化学习与蒙特卡洛树搜索的量子布尔函数综合方法：资源约束搜索视角

中文论文稿 v35: Affine-FPRM phase search 版

2026 年 7 月 8 日

摘要

量子布尔函数 oracle 综合把经典谓词嵌入量子算法，是 Grover 搜索、量子密码分析和约束求解类量子程序中的基础编译环节。现有方法通常从固定布尔表示、可逆模板、LUT 映射、XAG 网络或 CNOT 导向空间结构出发；这些路线能够在局部指标上取得效果，但很难同时处理 T-count、CNOT、深度、门数和辅助比特之间的组合权衡。本文将 Boolean oracle synthesis 重新表述为代数正规形 (algebraic normal form, ANF) 和固定极性 Reed-Muller (fixed-polarity Reed-Muller, FPRM) 项集合上的资源约束搜索问题，并提出 Resource-NMCTS：一种结合神经动作先验、蒙特卡洛树搜索、布尔环线性因子筛选、结构深度策略、depth-frontier policy 与保守 guard 的逻辑层综合框架。本文方法不依赖 SSHR 的 parallelotope 候选空间；SSHR 仅作为 small-scale CNOT-oriented baseline。

在 $n \leq 6$ 的 177 个传统函数上，Pareto-Resource-NMCTS 相对 direct ANF 平均 T-count 降低 72.25%，平均 score 降低 67.80%；相对 ESOP cube beam 获得 174/0/3 的 score 胜/负/平，平均 score 降低 36.09%；相对 weighted ESOP-MILP 获得 167/3/7，平均 score 降低 29.84%。在外部工具链 probe 中，ROS-style LUT proxy 覆盖 309 个函数和 927 个 ABC 映射行，best- K proxy 相对固定 $K = 4$ 平均 score 降低 18.12%，但 Pareto-Resource-NMCTS 相对该更强 proxy 仍为 309/0/0，平均 score 降低 84.27%。官方 mockturtle KLUT-to-XAG probe 在传统 177 个函数上给出 166/11/0、平均 score 降低 31.50%，在 $n = 14$ 的 64 个高维函数上给出 64/0/0、平均 score 降低 91.49%。高维正确性通过两层桥接补强： $n = 20-40$ 项集级 6600/6600 个方法行通过 ANF plan 展开验证和 emitted circuit 符号验证； $n = 21, 22, 23$ 基础 bridge 及两组 $n = 23$ policy rerun 合计 280/280 个方法行通过完整 oracle 验证。本文进一步用位掩码 truth-table evaluator 将完整 bridge 推进到 $n = 24, 25$ ：新增两个 6 函数切片共 120/120 个方法行同时通过完整 oracle、plan ANF 和 emitted-circuit 符号验证，其中 $n = 25$ 平均 truth-table 构造时间为 0.45 s/函数。至此本文累计报告 400/400 个高维 bridge 方法行的完整 oracle 验证。

本文同时报告 gate-set 边界。RevKit Python `oracle_synth` 在 177 个函数上全部返回，但 171/177 行含非 Clifford R_z ，总数为 9242，因此其 lower-bound phase netlist 不能直接当作精确 Clifford+T T-count 对比。为把该边界推进到项目内部，本文实现 phase-parity ANF emitter：将每个 ANF 单项式精确展开为 parity-phase gadgets，177/177 个函数均验证为 phase oracle up to global phase。该 emitter 相对 RevKit lower-bound score 为 40/137/0、平均高 69.25%，但在每个非 Clifford R_z 加 1 个 score 单位后转为 177/0/0、平均降低 48.16%，在 $T/R_z = 30$ 近似旋转综合 proxy 下为 177/0/0、平均降低 64.98%。进一步地，本文把 phase search 从 fixed-polarity FPRM 扩展到 Affine-FPRM：先在有界可逆线性变换 B 下综合 $h(y) = f(B^{-1}y)$ ，再枚举极性 p 得到 $g(z) = h(z \oplus p)$ ，并把选中的 parity mask 精确翻译回原变量。Affine-FPRM 三个 rank metric 共 531 行全部验证通过；其中 $T/R_z = 30$ 目标在 81/177 个函数选择非 identity 线性变

换, 相对 fixed-polarity FPRM 为 81/0/96、平均 score 降低 2.51%, 相对 phase-parity ANF 为 85/0/92、平均降低 2.98%, 相对 RevKit $T/R_z = 30$ proxy 为 177/0/0、平均降低 65.50%。这说明 phase/Rz 分支已经从成本敏感性和极性枚举推进到可验证的仿射代数预条件搜索; 但它仍是逻辑层 phase-oracle emitter, 不输出近似旋转序列, 也不包含硬件 mapping。阶段性结论是: Resource-NMCTS 已经形成一条独立于 SSHR、以低 T-count 与低加权逻辑资源为目标的 Boolean oracle synthesis 主线; 当前缺口是更强的学习式 phase/Rz-aware search、官方 ROS 或 legacy RevKit/CirKit 完整复现, 以及投稿级图表与论证收敛。

关键词: 量子 oracle 综合; 布尔函数综合; 强化学习; 蒙特卡洛树搜索; ANF; FPRM; 资源约束编译

1 本文定位: 把 oracle 综合写成搜索问题

本文的核心问题是: 给定布尔函数 $f: \{0,1\}^n \rightarrow \{0,1\}$, 如何自动生成实现

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle \quad (1)$$

的逻辑层可逆线路, 并在 T-count、CNOT、深度、门数和辅助比特之间取得更好的资源权衡。这个问题本质上不是单一规则匹配问题, 而是组合搜索问题: 同一个函数可以有許多等价的布尔表示、因子分解、极性选择和计算/反计算顺序, 不同选择会导致完全不同的量子逻辑资源。

本文不从 SSHR 入手。SSHR 的贡献在于利用 parallelotope 空间结构做 small-scale CNOT-oriented synthesis[6], 适合作为 CNOT 导向 baseline; 但本文要解决的是更一般的资源约束搜索问题。换言之, SSHR 在本文中回答的是“CNOT-oriented small-scale baseline 怎么样”, 而不是决定本文方法空间。本文的状态、动作、验证和资源模型都定义在 ANF/FPRM 项集合及其代数分解上。

本文主张可以压缩为一句话:

在逻辑层 bit-flip Boolean oracle synthesis 中, 用 ANF/FPRM 项集合表示状态、用可反算的代数分解定义动作, 并用神经先验和 MCTS 控制搜索, 可以在传统函数、高维项集合和若干外部逻辑网络 probe 上降低 T-count 与加权 score; 该主张不覆盖 phase/Rz oracle 最优性、硬件 routing、magic-state factory、噪声模型或后端映射最优性。

这个边界是本文能否投稿的关键。若把文章写成“SSHR 的改进”, 贡献会被限制在 small-scale CNOT-oriented synthesis; 而从搜索问题出发, 本文可以自然比较 SSHR、ESOP、CNOT、T-count、辅助比特、线路长度、深度和 phase/Rz gate-set 边界。AI 的角色也更清楚: 它不是直接生成无约束门序列, 而是在可验证动作空间中分配搜索预算。

表 1: 本文固定术语与使用范围。

术语	本文含义
Resource-NMCTS	在 ANF/FPRM 项集合上进行资源加权搜索的主框架, 包括神经动作先验、MCTS、结构策略、guard 和 portfolio 选择。
Boolean-ring screen	在 square-free Boolean ring 中搜索可复用线性因子的结构筛选分支, 允许线性因子与 quotient 共享变量。
Depth-frontier policy	在 depth-2、depth-3 和 depth-4 screen 之间选择的高预算质量-时间折中策略。
Guard	以确定性 baseline 为兜底的保守门控; 只有学习候选不劣于 baseline 时才接受。
Truth-table bridge	在 $n = 21, 22, 23, 24, 25$ 构造完整 truth-table, 把大规模项集符号验证和函数级完整验证连接起来。
SSHR	CNOT-oriented small-scale baseline; 本文方法不使用 SSHR parallelotope 候选空间。

2 相关工作与差异

2.1 布尔表示与低 T-count oracle 综合

低 T-count oracle 综合长期依赖合适的布尔中间表示。Xor-And-Inverter Graphs (XAG) 把 XOR 和 AND 结构分离, 使 AND 节点数与非 Clifford 成本形成清晰联系。Meuli 等人讨论了 multiplicative complexity 在低 T-count oracle 编译中的作用 [1], 并将 XAG 用于量子编译 [2]。ROS 将 oracle synthesis 写成 resource-constrained LUT mapping[3], 进一步表明 LUT 分解、局部函数实现和辅助比特管理会共同影响资源。BDD-based reversible synthesis 展示了面向大函数的符号结构路线 [4]。后端感知 oracle synthesis 则开始把逻辑综合和 fault-tolerant back-end 指标连接起来 [5]。

本文与这些工作的共同点是承认布尔中间表示决定量子逻辑资源。差异在于, 本文不固定使用 XAG、LUT 或 BDD, 而是以 ANF/FPRM 项集合为搜索状态。这样做的优势是任意候选 plan 都能展开回 $GF(2)$ 多项式进行等价检查, 且动作可以包含公共因子、极性重写、线性奇偶因子和布尔环共享变量结构。代价是需要重新设计搜索策略和外部对照解释, 尤其要避免把逻辑层 proxy 写成硬件级结论。

2.2 学习式搜索与线路综合

学习式搜索已经进入经典电路设计、量子线路综合和图重写优化。ShortCircuit 使用 AlphaZero 风格搜索进行经典布尔电路设计 [7]; Gumbel AlphaZero 被用于 Clifford+T unitary synthesis[8]; 强化学习和图神经网络也被用于 ZX-calculus rewrite-rule 选择 [9]; MonteQ 使用 MCTS 处理量子线路综合中的动作排序问题 [10]。这些研究说明, 离散综合中的动作空间往往过大, 学习策略可以帮助搜索更快进入高质量区域。

本文的状态和动作与上述工作不同。我们不让模型直接输出门序列, 而是让模型在可验证的代数动作之间排序或选择。最终线路由 compute/action/uncompute 结构生成, 并经过 ANF 展开、

GF(2) 多项式模拟或完整 truth-table 验证。这样的设计降低了 AI 产生错误线路的风险，也使实验可以清楚区分“学习策略带来的搜索改进”和“确定性代数结构本身的收益”。

3 问题定义与资源模型

本文输入可以是完整 truth-table，也可以是 ANF 项集合。对于

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i, \quad (2)$$

$\mathcal{M}$ 表示 square-free monomial 集合。输出线路使用 X、CNOT 和多控 Toffoli (MCT) 抽象门，并在逻辑层估计 T-count、CNOT、depth、gate count 和 peak ancilla。搜索排序使用统一分数

$$\text{score} = T + 0.04 \text{ CNOT} + 0.015 \text{ depth} + 0.01 \text{ gates} + 2 \text{ ancilla}. \quad (3)$$

该分数只用于逻辑层候选排序，不代表真实硬件运行时间、物理错误率或 magic-state factory 总成本。因此，实验同时报告 T、CNOT、深度和辅助比特，避免单一 score 掩盖 trade-off。

正确性验证分为三层。第一层在 $n \leq 6$ 传统函数、 $n = 18, 20$ 函数级切片和 $n = 21, 22, 23, 24, 25$ bridge 中构造完整 truth-table，逐点验证式 (1)。第二层在大规模项集实验中，将 factor plan 展开回 ANF 项集合，检查目标多项式是否一致。第三层对 emitted X/CNOT/MCT circuit 做 GF(2) 多项式符号模拟，检查输出线多项式、输入线复原和辅助线清零。第三层不是 2^n 枚举，但验证的是最终 emitted circuit，而不只是搜索 plan。

为使 $n = 24, 25$ 完整枚举可运行，本文在验证 harness 中加入位掩码 truth-table evaluator。对变量 x_i 预先构造赋值位集 V_i ，单项式 $m = \prod_{i \in S} x_i$ 的真值表为

$$T_m = \bigwedge_{i \in S} V_i, \quad (4)$$

常数项对应全 1 掩码，完整 ANF 真值表则由所有 T_m 异或得到。这个实现只改变完整验证的构造方式，不改变搜索算法或资源统计；小规模随机 ANF 已与原 zeta 实现逐项对比通过。

4 方法

4.1 项集合搜索状态

Resource-NMCTS 的状态是当前尚未综合的项集合 $\mathcal{M}$ 。一次动作选择一个可计算、可反算的代数结构，把 $\mathcal{M}$ 分解为 quotient 子问题和 rest 子问题，再递归生成线路。候选动作包括公共单项式因子、FPRM 极性重写后的公共项、线性奇偶因子、布尔环线性因子和由神经模型扩展的 root actions。动作被接受后，框架会估计即时资源收益、子问题大小、辅助比特峰值和后续可分解性。

神经动作先验用于对候选排序，MCTS 在有限预算内探索候选组合，rollout 使用确定性资源估计和已有 baseline 候选。最终 portfolio 在多个候选线路中按式 (3) 选择最优者。这个流程的关键不是“模型足够聪明”，而是每个候选动作都可以回到 GF(2) 语义验证；模型只影响搜索顺序和预算分配，不改变正确性定义。

4.2 布尔环线性因子

传统线性因子筛选通常要求线性因子变量和 quotient 变量不相交。这个限制便于实现，但会排除 square-free Boolean ring 中合法的重叠结构。本文的 Boolean-ring screen 允许二者共享变量，并在展开时使用 $x_i^2 = x_i$ 与 GF(2) 偶数项消去。例如

$$(x_i \oplus x_j)x_im = x_im \oplus x_ix_jm. \quad (5)$$

线路层面仍可用 CNOT 计算线性奇偶量，再以该辅助量控制 quotient 子线路。这样，screen 不只是把 beam 宽度调大，而是把原先被实现约束排除的代数候选重新放回可综合空间。

4.3 结构策略、frontier policy 与 guard

本文把 AI 模块拆成三类。第一类是动作排序，模型根据候选覆盖率、项数变化、变量覆盖密度、次数分布和即时资源估计对候选动作打分。第二类是结构选择，depth policy 判断当前项集合适合 single、depth-1 还是 depth-2 screen。第三类是 frontier 选择，depth-frontier policy 在 depth-2、depth-3 和 depth-4 screen 之间选择。质量型 frontier policy 用 score-only oracle frontier 训练，目标是在接近 all-depth frontier 的质量时减少全深度枚举开销；cost-aware frontier policy 则在标签中加入相对运行时间惩罚，用小幅 score 代价换取更低 plan time 和更低辅助线 lifetime area。

Guard 用来限制学习策略风险。若学习候选相对确定性 baseline 出现 score 退化，则返回 baseline。Depth-2 skip guard 避免把应该运行 depth-2 的项集合错误跳过。Screen-gate 用于判断某些边界函数是否可以跳过昂贵的 Resource tail。这样的设计让 AI 贡献可以表现为质量提升、运行时间节省或质量-时间折中，而不是要求学习策略在所有切片上无条件支配确定性 baseline。

表 2: Resource-NMCTS 的模块与证据钩子。

模块	机制	证据钩子
ANF/FPRM 搜索状态	把 oracle 综合写成项集合分解问题，所有候选可回到 GF(2) 多项式验证。	小规模 truth-table 验证与项集级 ANF 验证。
Boolean-ring screen	搜索允许共享变量的线性因子，递归处理 quotient/rest 子问题。	$n = 18, 20$ 函数级边界和 $n = 20-40$ scale。
神经动作先验与 MCTS	对候选动作排序并在有限预算内组合分解动作。	传统函数上的搜索消融和 portfolio 改善。
Depth-frontier policy	学习 depth-2/3/4 质量前沿，提供质量模式与 cost-aware 模式。	独立 seed 泛化和 $n = 21, 22, 23, 24, 25$ bridge。
Guard	用 baseline-preserving 规则兜底。	无 score-loss guard 与运行时节省。

5 实验设置

实验回答三个问题。第一，Resource-NMCTS 在完整 truth-table 可验证的小规模函数上是否确实降低 T-count 和 score。第二，在高维函数和项集合上，结构筛选、frontier policy 和 guard 是

否仍能运行并保持语义正确。第三，与外部逻辑网络或 phase/Rz 工具链相比，本文优势是否仍成立，或暴露出哪些 gate-set 边界。

传统函数切片包含 $n \leq 6$ 的 177 个函数，baseline 包括 direct ANF、AND-direct ANF、ESOP cube beam、weighted ESOP-MILP、SSHR-H、ABC/BDD 估计、ROS-style LUT proxy、mockturtle KLUT-to-XAG probe 和 RevKit Python `oracle_synth`。高维切片包含 $n = 14, 15, 16, 18$ 的外部导出函数、 $n = 18, 20$ 函数级边界、 $n = 20-40$ 项集级 scale，以及 $n = 21, 22, 23, 24, 25$ truth-table bridge。Paired comparison 只纳入函数名或项集名匹配、语义正确且未 timeout 的结果。Timeout 行作为可扩展性边界保留，但不进入资源均值。

表 3: 实验层次与回答的问题。

实验层次	数据与对照	回答的问题
传统函数	$n \leq 6$, 177 个函数; ESOP、SSHR、ABC/BDD、mockturtle probe、RevKit API。	小规模完整验证条件下是否形成低 T-count 与低 score 优势。
ROS-style LUT proxy	ABC $K = 3, 4, 5$ sweep, 309 个函数, 927 行检查。	更强 LUT proxy 是否削弱本文优势。
mockturtle XAG probe	ABC $K = 4$ BLIF 到官方 mockturtle KLUT-to-XAG resynthesis。	official-header probe 是否可运行，以及本文是否仍优于该 XAG proxy。
高维函数边界	$n = 18, 20$ 函数级切片。	高维完整 truth-table 条件下哪些搜索分支仍有效。
项集级 scale	$n = 20-40$, 6600 个方法行。	结构策略是否可扩展，以及 plan 和 emitted circuit 是否符号等价。
Truth-table bridge	$n = 21, 22, 23$ 基础 bridge 及两组 $n = 23$ policy rerun 合计 280 个方法行，加 $n = 24, 25$ 各 60 个方法行完整验证。	项集级符号验证和函数级 oracle 验证是否一致，以及完整验证边界能否继续上移。

6 结果

6.1 传统函数上的资源优势

表 4 给出 $n \leq 6$ 传统函数上的平均逻辑资源。Pareto-Resource-NMCTS 相对 direct ANF 平均 T-count 降低 72.25%，平均 score 降低 67.80%；相对 ESOP cube beam 获得 174/0/3 的 score 胜/负/平，平均 score 降低 36.09%；相对 weighted ESOP-MILP 获得 167/3/7，平均 score 降低 29.84%。SSHR-H 的平均 CNOT 最低，因此本文不能声称 CNOT-only 支配 SSHR。更稳妥的结论是，本文用一定 CNOT 和辅助比特 trade-off 换取更低 T-count、depth 和加权 score。

表 4: $n \leq 6$ 传统函数切片上的平均逻辑资源。

方法	平均 T	平均 CNOT	平均 depth	平均 ancilla	平均 score
Direct ANF	184.1	134.2	134.6	1.33	194.3
AND-direct ANF	101.0	166.5	166.9	2.18	115.2
Resource-NMCTS	43.9	89.9	94.5	1.92	53.2
Pareto-Resource-NMCTS	40.4	83.0	88.0	2.03	49.6
ESOP-MILP	83.6	133.5	159.6	2.32	96.7
SSHR-H	81.0	67.6	88.7	1.36	88.2

6.2 外部逻辑网络 probe

ROS-style LUT proxy 先用 ABC 对导出 BLIF 运行 $K = 3, 4, 5$ sweep, 再把每个映射后的 LUT 网络按 local ANF compute/action/uncompute 方式估计逻辑资源。该 proxy 覆盖 309 个函数, 共 927 个映射行, 全部通过 truth-table 检查。Best- K proxy 相对固定 $K = 4$ 自身获得 219/0/90 的 score 胜/负/平, 平均 score 降低 18.12%, 说明它确实比单一 K 设置更强。在这个更强 baseline 上, Resource-NMCTS 和 Pareto-Resource-NMCTS 均在 309/309 个函数上取得更低 score, 平均 score 分别降低 83.77% 和 84.27%。

mockturtle probe 进一步把外部比较从 ABC-LUT 内部 proxy 推进到官方 mockturtle 头文件生态。流程为: 先用 ABC 将导出 BLIF 映射为 $K = 4$ LUT 网络, 再用 mockturtle `blif_reader` 读入 KLUT network, 并调用 `xag_npn_resynthesis` 重综合为 XAG。该流程不是官方 ROS, 不包含 SAT garbage management, 也不输出可逆 Clifford+T 线路; 它的作用是给出一个可运行、可编译、可统计的 official-header KLUT-to-XAG probe。

表 5: 官方 mockturtle KLUT-to-XAG probe 对比。资源为 XAG AND/XOR/depth 派生的逻辑层 proxy, 不是官方 ROS、可逆 garbage management 或硬件 mapping。

Target vs mockturtle XAG K4	Items	W/L/T	Mean Δ
and_resource_nmcts	177	165/12/0	-28.97%
and_pareto_resource_nmcts	177	166/11/0	-31.50%
and_fprm_polarity_archive	177	156/21/0	-25.11%
and_direct_anf	177	19/158/0	+50.20%
direct_anf	177	9/168/0	+142.26%
and_esop_milp	177	92/85/0	+8.18%
sshr_h	177	50/127/0	+33.30%

Target vs mockturtle XAG K4	Items	W/L/T	Mean Δ
and_resource_nmcts	64	64/0/0	-91.41%
and_profile_resource_nmcts	64	64/0/0	-91.41%
and_pareto_resource_nmcts	64	64/0/0	-91.49%
and_direct_anf	64	64/0/0	-88.40%
direct_anf	64	64/0/0	-82.58%

表 5 显示, 在 $n \leq 6$ 的 177 个传统函数上, mockturtle probe 177/177 行正确、无 error 或 timeout; Pareto-Resource-NMCTS 相对该 probe 为 166/11/0, 平均 score 降低 31.50%。在 $n = 14$ 的 64 个高维函数上, probe 64/64 行正确, Pareto-Resource-NMCTS 为 64/0/0, 平均 score 降低 91.49%。需要注意的是, 高维切片中本文方法的 depth proxy 可能高于 XAG probe; 主要优势来自 T、CNOT 和 peak ancilla 的大幅降低。因此这里的主张仍是加权逻辑资源优势, 而不是所有分项无条件支配。

6.3 RevKit phase/Rz 边界与内部 emitter

RevKit Python `oracle_synth` 在 $n \leq 6$ 的 177 个函数上全部返回, 但返回的是 Rz-phase netlist, 而不是可直接按 T/Tdg 完整计数的 Clifford+T netlist。角度审计显示, 只有 6 行不含非 Clifford R_z ; 171 行含非 Clifford R_z , 总数为 9242, 角度除以 π 的最大分母为 64。因此, Resource-NMCTS 相对 RevKit lower-bound score 的失败不能解释为精确 Clifford+T T-count 失败, 而应解

释为不同 gate-set/phase 口径下的压力测试。

表 6: RevKit phase/Rz 边界结果。

比较口径	胜/负/平	平均相对变化
Resource-NMCTS vs RevKit lower-bound score	6/171/0	+751.69%
Resource-NMCTS vs RevKit score+1/Rz	140/37/0	-14.52%
Resource-NMCTS family vs RevKit score+1/Rz	157/20/0	-17.89%
Resource-NMCTS family vs RevKit score+1.5/Rz	177/0/0	-42.26%
Resource-NMCTS family vs RevKit $T/R_z = 30$ proxy	177/0/0	-95.03%

为了避免 phase/Rz 讨论只停留在 RevKit 侧的成本敏感性, 本文进一步实现了一个内部 phase-parity ANF emitter。对每个非空 ANF 单项式 S , 它使用恒等式

$$\prod_{i \in S} x_i = \frac{1}{2^{|S|-1}} \sum_{\emptyset \neq T \subseteq S} (-1)^{|T|+1} \bigoplus_{i \in T} x_i \quad (6)$$

把单项式相位展开为 parity-phase gadgets, 并合并相同 parity mask 的旋转角。常数项被视为全局相位, 因此验证目标是 phase oracle up to global phase。该 emitter 在 $n \leq 6$ 的 177 个传统函数上全部通过 Fraction 精确验证, 平均 lower-bound score 为 10.11, 平均 CNOT 为 87.40, 平均 depth 为 114.96, 平均 R_z 总数为 27.56, 平均非 Clifford R_z 为 21.75, 最大角度分母为 32。

表 7: Phase-parity ANF emitter 与 RevKit `oracle_synth` 的 phase/Rz 同口径比较。该 emitter 实现 phase oracle up to global phase; lower-bound score 只计 T-like rotations, 非 Clifford R_z 单独报告。

Metric	Items	W/L/T	Mean Δ
lower-bound score	177	40/137/0	+69.25%
score+1/Rz	177	177/0/0	-48.16%
score+1.5/Rz	177	177/0/0	-53.26%
score+2/Rz	177	177/0/0	-56.05%
$T/R_z = 30$ proxy	177	177/0/0	-64.98%
non-Clifford R_z	177	171/0/6	-63.33%
total R_z	177	173/3/1	-45.10%

表 7 的结论并不是“naive phase expansion 已经解决问题”。相反, 它显示 naive parity-phase 展开在 RevKit lower-bound score 下只有 40/137/0, 平均 score 反而高 69.25%; 但它的非 Clifford R_z 数量相对 RevKit 为 171/0/6, 平均降低 63.33%, 总 R_z 数量为 173/3/1, 平均降低 45.10%。一旦每个非 Clifford R_z 加入 1 个 score 单位, 它相对 RevKit 转为 177/0/0, 平均 score 降低 48.16%; 在 $T/R_z = 30$ 近似旋转综合 proxy 下同样为 177/0/0, 平均降低 64.98%。因此, phase-parity baseline 的价值在于把 phase/Rz 边界从“RevKit 的成本重解释”推进为“项目内部已有可验证 emitter”, 同时也说明下一步必须做学习式或优化式 phase/Rz-aware search, 以减少 parity gadget 数和旋转谱, 而不是依赖朴素展开。

为了让 phase/Rz 方向真正具有搜索结构, 本文进一步加入 fixed-polarity phase-parity search。对每个极性 p , 先构造 $g(z) = f(z \oplus p)$, 计算 g 的 ANF, 再把每个 shifted parity mask 翻译回 x

变量。若 p 与 parity mask 的交叠为奇数，则

$$\bigoplus_{i \in T} z_i = 1 - \bigoplus_{i \in T} x_i, \quad (7)$$

因此对应旋转角取相反数，并把原角度并入全局相位。验证时不再只检查常数项全局相位，而是检查所有输入上的 phase difference 是否为同一个有理数。这个检查更适合 phase oracle，因为极性平移会产生非整数的全局相位。

表 8: Fixed-polarity phase-parity search. 每个函数枚举全部 2^n 个极性，按 lower-bound score、score+1/Rz 和 $T/R_z = 30$ proxy 三个目标分别选择最优极性；共 531 行全部 up-to-global-phase 验证通过。

Comparison	Items	W/L/T	Mean Δ
FPRM-score vs ANF	177	59/0/118	-3.98%
FPRM-Rz1 vs ANF	177	59/0/118	-1.65%
FPRM- $T/R_z = 30$ vs ANF	177	59/0/118	-0.47%
FPRM- $T/R_z = 30$ vs RevKit	177	177/0/0	-65.16%
FPRM non-Clifford Rz vs RevKit	177	171/0/6	-63.48%
FPRM CNOT vs RevKit	177	34/140/3	+34.18%

表 8 显示，FPRM 极性搜索相对原始 phase-parity ANF 没有产生负退化：按 lower-bound score 选择时为 59/0/118、平均降低 3.98%；按 score+1/Rz 选择时为 59/0/118、平均降低 1.65%；按 $T/R_z = 30$ proxy 选择时为 59/0/118、平均降低 0.47%。在 $T/R_z = 30$ 目标下，59/177 个函数选择非零极性，平均极性权重为 0.57，平均非 Clifford R_z 从 21.75 降到 21.60。这个增量是正向的，但幅度仍小；它说明极性搜索是合理的 phase/Rz 搜索维度，而不是已经形成足够强的 phase/Rz-aware 方法。

为进一步扩大 phase/Rz 搜索空间，本文新增 Affine-FPRM phase-parity search。它在 fixed-polarity 搜索前加入有界可逆线性预条件：对 GF(2) 可逆矩阵 B ，先定义

$$h(y) = f(B^{-1}y), \quad g(z) = h(z \oplus p), \quad (8)$$

再对 g 合成 phase polynomial。若该 polynomial 中某个 parity mask 为 m_z ，则 $z = Bx \oplus p$ 给出原变量上的 mask

$$m_x = B^T m_z. \quad (9)$$

当 $m_z \cdot p = 1$ 时，对应 parity 被取反，因此旋转角取相反数并把原角度并入全局相位。这个过程只改变 phase polynomial 的代数表示，并不把 B 当作额外硬件 CNOT wrapper 计入；验证仍逐输入检查 phase difference 是否为同一个有理数。搜索中包含 identity transform，因此在同一 rank metric 下 Affine-FPRM 相对 fixed-polarity FPRM 不退化。

表 9: Affine-FPRM phase-parity search 的资源比较。每个函数搜索至多 32 个可逆线性变换，并在每个变换下枚举全部极性；共 531 行 selected results 全部 up-to-global-phase 验证通过。

Comparison	Items	W/L/T	Mean Δ
Affine-FPRM- $T/R_z = 30$ vs FPRM	177	81/0/96	-2.51%
Affine-FPRM- $T/R_z = 30$ vs ANF	177	85/0/92	-2.98%
Affine-FPRM- $T/R_z = 30$ vs RevKit	177	177/0/0	-65.50%
Affine-FPRM non-Clifford R_z vs FPRM	177	20/0/157	-0.73%
Affine-FPRM-score vs FPRM	177	80/0/97	-5.77%

表 9 显示，Affine-FPRM 是当前 phase/ R_z 分支中比 fixed-polarity 更强的证据。在 $T/R_z = 30$ 目标下，它有 81/177 个函数选择非 identity 线性变换、43/177 个函数选择非零极性；相对 fixed-polarity FPRM 为 81/0/96、平均 score 降低 2.51%，相对 phase-parity ANF 为 85/0/92、平均降低 2.98%，相对 RevKit 为 177/0/0、平均降低 65.50%。按 lower-bound score 选择时，Affine-FPRM 相对 fixed-polarity FPRM 为 80/0/97、平均 score 降低 5.77%。同时它仍不是完整 phase/ R_z synthesis：当前报告的是 parity-gadget 级逻辑 emitter 与 T/R_z proxy，不包含近似旋转序列生成、旋转综合运行时或硬件 mapping。

6.4 高维结构策略与验证桥接

在 $n = 18$ 的 12 个随机 ANF 函数上，adaptive depth-2 Boolean-ring screen 相对 single screen 获得 11/0/1，平均 score 从 11349.06 降至 10670.94；完整 Resource-NMCTS 平均 score 为 7315.62，相对 adaptive screen 进一步降低 23.85%。在 $n = 20$ 边界函数上，FPRM root beam 和 fast linear-pair 均触发 300 s task timeout；adaptive depth-2 screen 相对 single screen 获得 5/0/1，平均 score 降低 7.47%；screen-gated Resource-NMCTS 可把平均时间从 77.10 s 降至 20.71 s。这个结果说明高维中结构筛选是主要质量来源之一，Resource tail 的收益取决于切片和预算。

Depth-frontier policy 将高预算 depth-2/3/4 质量前沿学习化。在 $n = 20, 28, 40$ scale harness 中，它相对 fixed depth-2 获得 35/0/37，平均 score 降低 2.19%；相对 all-depth adaptive depth ≤ 4 平均 score 只高 0.97%，但节省 58.69% 时间。扩大训练集后的 large frontier policy 在独立 seed $n = 24, 28, 32, 40$ 泛化集上相对 fixed depth-2 获得 56/0/40，平均 score 降低 2.34%；相对 all-depth adaptive 只高 0.10%，仍节省 53.50% 时间。Cost-aware frontier policy 则以 +0.99% score 代价换取相对 large policy 的 33.02% plan time 下降和 7.61% 辅助线 lifetime area 下降。

宽度消融进一步说明，单纯扩大 root action width 不是当前高维收益瓶颈。在 $n = 20, 28, 40$ 、每个宽度 72 个项集的 probe 中，action-width 取 6、12、24 时每组 504/504 方法行都通过 plan 和 emitted-circuit 符号验证；adaptive all-depth 相对 fixed depth-2 始终为 0/0/72，depth-frontier policy 相对 all-depth adaptive 只高 0.08% 左右，但随着宽度从 6 增至 24，全深度枚举相对 depth-2 的额外时间从 +57.52% 降到 +43.04%，而相对 single screen 的绝对开销从约 +3284% 增至 +23918%。这个结果支持本文默认使用 width 6 的结论：提升主要来自递归深度和 frontier 选择，而不是盲目加宽候选。

新增 $n = 24$ 和 $n = 25$ bridge 均使用式 (4) 的位掩码 truth-table evaluator。 $n = 24$ 在 6 个生成式 ANF 函数上产生 60 个方法行，全部通过完整 truth-table oracle 验证、plan ANF 展开验证和

emitted-circuit 符号验证, mismatch 为 0。表 10 显示, 在该完整枚举切片上, adaptive all-depth 相对 single screen 为 6/0/0、平均 score 降低 9.42%; 相对 fixed depth-2 为 4/0/2、平均 score 降低 2.21%。Depth-frontier policy 相对 fixed depth-2 为 3/0/3、平均 score 降低 2.05%; 相对 depth-4 只高 0.16%, 但 plan time 降低 27.08%。

$n = 25$ bridge 再次在 6 个生成式 ANF 函数上产生 60 个方法行, 三层验证同样全部通过, 平均 truth-table 构造时间为 0.45 s/函数。表 11 显示, adaptive all-depth 相对 single screen 为 6/0/0、平均 score 降低 7.31%; 相对 fixed depth-2 为 4/0/2、平均 score 降低 2.14%。Depth-frontier policy 相对 fixed depth-2 同样为 4/0/2、平均 score 降低 2.14%; 相对 depth-4 为 0/0/6, score 完全持平并节省 8.87% plan time; 相对 all-depth adaptive 也为 0/0/6, score 持平并节省 46.09% plan time。该结果把完整验证边界从 $n = 24$ 继续推到 $n = 25$, 同时表明 frontier policy 可以在更高维 bridge 上直接选择到质量前沿。

表 10: $n = 24$ 完整 truth-table bridge 的资源比较。

n	Method	Baseline	Score W/L/T	Mean Δ score	Mean Δ plan time
24	adaptive_all_depth	screen_single	6/0/0	-9.42%	+34710.06%
24	adaptive_all_depth	screen_depth2	4/0/2	-2.21%	+1070.36%
24	depth_policy	screen_single	6/0/0	-6.96%	+2155.21%
24	depth_frontier_policy	screen_depth2	3/0/3	-2.05%	+555.27%
24	depth_frontier_policy	screen_depth4	0/1/5	+0.16%	-27.08%
24	depth2_guard_direct	screen_depth2	0/0/6	+0.00%	+0.00%
24	screen_depth3	screen_depth2	4/0/2	-1.45%	+194.81%
24	screen_depth4	screen_depth2	4/0/2	-2.21%	+642.76%
all	adaptive_all_depth	screen_single	6/0/0	-9.42%	+34710.06%
all	adaptive_all_depth	screen_depth2	4/0/2	-2.21%	+1070.36%
all	depth_policy	screen_single	6/0/0	-6.96%	+2155.21%
all	depth_frontier_policy	screen_depth2	3/0/3	-2.05%	+555.27%
all	depth_frontier_policy	screen_depth4	0/1/5	+0.16%	-27.08%
all	depth2_guard_direct	screen_depth2	0/0/6	+0.00%	+0.00%
all	screen_depth3	screen_depth2	4/0/2	-1.45%	+194.81%
all	screen_depth4	screen_depth2	4/0/2	-2.21%	+642.76%

表 11: $n = 25$ 完整 truth-table bridge 的资源比较。

n	Method	Baseline	Score W/L/T	Mean Δ score	Mean Δ plan time
25	adaptive_all_depth	screen_single	6/0/0	-7.31%	+39171.94%
25	adaptive_all_depth	screen_depth2	4/0/2	-2.14%	+1215.08%
25	depth_policy	screen_single	6/0/0	-5.34%	+2393.41%
25	depth_frontier_policy	screen_depth2	4/0/2	-2.14%	+744.84%
25	depth_frontier_policy	screen_depth4	0/0/6	+0.00%	-8.87%
25	depth2_guard_direct	screen_depth2	0/0/6	+0.00%	+0.00%
25	screen_depth3	screen_depth2	4/0/2	-1.34%	+211.63%
25	screen_depth4	screen_depth2	4/0/2	-2.14%	+763.25%
all	adaptive_all_depth	screen_single	6/0/0	-7.31%	+39171.94%
all	adaptive_all_depth	screen_depth2	4/0/2	-2.14%	+1215.08%
all	depth_policy	screen_single	6/0/0	-5.34%	+2393.41%
all	depth_frontier_policy	screen_depth2	4/0/2	-2.14%	+744.84%
all	depth_frontier_policy	screen_depth4	0/0/6	+0.00%	-8.87%
all	depth2_guard_direct	screen_depth2	0/0/6	+0.00%	+0.00%
all	screen_depth3	screen_depth2	4/0/2	-1.34%	+211.63%
all	screen_depth4	screen_depth2	4/0/2	-2.14%	+763.25%

表 12: 高维验证桥接与结构策略结果。

验证项或比较	结果	说明
项集级 plan ANF 验证	6600/6600	覆盖 $n = 20$ 到 40 的 scale/frontier/frontier-policy rerun 与泛化 run。
项集级 emitted-circuit 符号验证	6600/6600	GF(2) 多项式模拟通过, mismatch 总数为 0。
$n = 21, 22, 23$ 完整 truth-table oracle 验证	280/280	包含基础 bridge 及两组 $n = 23$ policy rerun; 所有 emitted circuit 逐点验证通过。
$n = 24$ 完整 truth-table oracle 验证	60/60	平均 truth-table 构造时间 0.18 s/函数; plan 与 emitted-circuit 符号验证也均为 60/60。
$n = 25$ 完整 truth-table oracle 验证	60/60	平均 truth-table 构造时间 0.45 s/函数; depth-frontier policy 与 depth-4 score 持平且 plan time 降低 8.87%。
Large frontier vs fixed depth-2	56/0/40, score -2.34%	独立 seed 泛化集。
Large frontier vs all-depth adaptive	score $+0.10\%$, time -53.50%	质量接近全深度前沿但时间更低。
Cost-aware frontier vs large frontier	score $+0.99\%$, time -33.02% , aux area -7.61%	快速质量折中模式。

7 讨论：什么算明显提升

本文当前已经有一条独立于 SSHR 的主线: ANF/FPRM 项集合给出可验证状态, Boolean-ring screen 提供结构候选, 神经先验和 MCTS 负责搜索控制, frontier policy 和 guard 处理质量-时间折中与风险控制。这个主线比“改 SSHR”更适合作为论文方向, 因为它可以自然对比 SSHR、ESOP、CNOT、T-count、辅助比特、线路长度、深度和 phase/Rz 口径。

但这还不是最终投稿形态。当前最强证据是逻辑层 bit-flip oracle synthesis 的低 T-count 与低 score; 仍需继续补强的是 phase/Rz-aware search、标准外部工具链复现和投稿级结果组织。RevKit、phase-parity ANF、fixed-polarity phase search 与 Affine-FPRM 共同说明, gate-set 口径会显著改变结论。现在 phase/Rz 方向已经不只是成本敏感性分析: 它有一个可验证 emitter、一个可验证极性搜索分支, 以及一个可验证的仿射线性预条件分支; 其中 Affine-FPRM 相对 fixed-polarity FPRM 已给出 81/0/96、平均 score 降低 2.51% 的 $T/R_z = 30$ proxy 增益。这个结果足以证明 phase/Rz 可以写成代数搜索问题, 但仍不足以声明 phase/Rz 全局最优。 $n = 24, 25$ bridge 则说明正确性验证能力可以继续上移, 但它们仍是小样本完整枚举, 不等同于 $n = 26-40$ 全部完成 truth-table simulation。

因此, 本文的“明显提升”目标应设置为三个可检查终点。第一, 把 Affine-FPRM phase search 进一步扩展为学习式或优化式 phase/Rz-aware policy, 在 177 个传统函数上保持不退化, 并继续

降低 parity gadget 数、非 Clifford R_z 数或综合后 proxy score, 同时保持 selected rows 全部 up-to-global-phase 验证。第二, 继续复现官方 ROS 或 legacy RevKit/CirKit 完整流程, 使外部对照从 adapter/probe 扩展为标准工具链。第三, 把当前 v35 的阶段性报告重组为投稿稿: 围绕 4–6 张主图和若干补充表组织主张、baseline、公平性、消融和限制, 而不是继续以实验日志形式累积结果。硬件 mapping 不属于本文的目标范围, 本文只报告逻辑层资源和逻辑层 schedule/lifetime proxy。

表 13: 当前主张、证据与边界。

主张	当前证据	边界
本文路线独立于 SSHR	状态、动作和验证都定义在 ANF/FPRM 项集合上; SSHR 只作为 baseline。	不能声称 CNOT-only 支配 SSHR。
低 T-count 与低 score 是主优势	$n \leq 6$ 传统函数相对 ANF、ESOP 和 weighted ESOP-MILP 有稳定优势。	资源权重仍是逻辑层人为设定。
外部逻辑网络 probe 下仍有优势	ROS-style LUT proxy 为 309/0/0; mockturtle traditional 为 166/11/0, 高维 $n = 14$ 为 64/0/0。	仍不是官方 ROS 或可逆线路输出。
Phase/Rz 边界已有实际 emitter 与 Affine-FPRM 搜索	RevKit 171/177 行含非 Clifford R_z ; phase-parity ANF emitter 177/177 验证通过; fixed-polarity 与 Affine-FPRM phase search 均 531/531 验证通过; Affine-FPRM 的 $T/R_z = 30$ 目标相对 fixed-polarity FPRM 为 81/0/96、相对 ANF 为 85/0/92、相对 RevKit 为 177/0/0。	仍是 parity-gadget 级逻辑 emitter 与 proxy score; 需要 learned phase/Rz policy 和实际旋转序列审计。
高维结果具有语义可信度	6600/6600 项集级符号验证和 400/400 bridge 完整验证通过, 其中 $n = 24$ 与 $n = 25$ 各为 60/60。	$n = 26-40$ 未做完整 truth-table 枚举。

8 结论

本文提出 Resource-NMCTS, 将量子布尔函数 oracle 综合建模为 ANF/FPRM 项集合上的资源约束搜索问题, 并结合神经动作先验、MCTS、布尔环线性因子筛选、结构深度策略、depth-frontier policy 和保守 guard 生成逻辑层可逆线路。实验表明, 该方法在传统函数上相对 ANF、ESOP、SSHR-H 和 weighted ESOP-MILP 获得稳定低 T-count 与低 score 优势, 在 ROS-style LUT proxy 和官方 mockturtle KLUT-to-XAG probe 下仍保持优势, 并通过项集级符号验证和 $n = 21, 22, 23, 24, 25$ 完整 truth-table bridge 补强了高维正确性证据。

当前稿件最适合定位为“资源约束量子布尔函数综合的搜索方法论文”。它不是 SSHR 的附属改进, 也不是泛泛使用 AI 做线路生成; 它的核心贡献是把 oracle synthesis 的动作空间、搜索策略、资源模型和语义验证放在同一个 ANF/FPRM 框架中。v32 的 fixed-polarity phase-parity search 说明 phase/Rz 方向已经进入可验证搜索阶段, v35 的 Affine-FPRM 搜索进一步把该分支推进到线性预条件和极性的联合代数搜索; v33–v34 将高维完整 oracle 验证边界从 $n = 24$ 推进到 $n = 25$, 并用 action-width probe 说明扩宽候选不是当前主收益来源。下一步若能把 learned phase/Rz policy、标准外部工具链复现和投稿级图表组织补齐, 本文就可以从阶段性方法证据推进到更强的投稿版

本。

参考文献

- [1] G. Meuli, M. Soeken, E. Campbell, M. Roetteler, and G. De Micheli. The role of multiplicative complexity in compiling low T-count oracle circuits. In *Proceedings of ICCAD*, 2019.
- [2] G. Meuli, M. Soeken, and G. De Micheli. Xor-And-Inverter Graphs for quantum compilation. *npj Quantum Information*, 8:7, 2022.
- [3] G. Meuli, M. Soeken, M. Roetteler, and G. De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.
- [4] R. Wille and R. Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of DAC*, pp. 270–275, 2009.
- [5] M. Yu, A. Tempia Calvino, M. Soeken, and G. De Micheli. Back-end-aware fault-tolerant quantum oracle synthesis. In *Proceedings of ASP-DAC*, pp. 930–937, 2025.
- [6] Q. Zheng, Y. Xu, J. Zhang, Z. Su, and S. Zheng. CNOT oriented synthesis for small-scale Boolean functions using spatial structures of parallelotopes. In *Proceedings of ICCAD*, 2025.
- [7] D. Tsaras, A. Grosnit, L. Chen, Z. Xie, H. Bou-Ammar, and M. Yuan. ShortCircuit: AlphaZero-driven circuit design. *arXiv:2408.09858*, 2024.
- [8] S. Rietsch, A. Y. Dubey, C. Ufrecht, M. Periyasamy, A. Plinge, C. Mutschler, and D. D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *IEEE International Conference on Quantum Computing and Engineering*, pp. 824–835, 2024.
- [9] J. Riu, J. Nogu , G. Vilaplana, A. Garcia-Saez, and M. P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.
- [10] M. Machiya, M. Menickelly, P. Hovland, and J. Liu. MonteQ: A Monte Carlo tree search based quantum circuit synthesis framework. *arXiv:2604.19029*, 2026.